LAB NO: 4

Objectives

To learn to use MATLAB to (1) generate an LTI state-space representation of a system and (2) convert an LTI state-space representation of a system to an LTI transfer function.

Two approaches are available for the analysis and design of feedback control systems. The first, introduced in the Modeling in the Frequency Domain chapter, is known as the ***classical***, or ***frequency-domain***, technique. This approach is based on converting a system's differential equation to a transfer function, thus generating a mathematical model of the system that *algebraically* relates a representation of the output to the input. Replacing a differential equation with an algebraic equation not only simplifies the representation of individual subsystems but also simplifies modeling interconnected subsystems. The primary disadvantage of the classical approach is limited applicability: only for linear, time-invariant systems or systems that can be approximated as such.

A major advantage of frequency-domain techniques: to rapidly provide stability and transient response information. Thus, we can immediately see the effects of varying system parameters until an acceptable design is met.

With the arrival of space exploration, requirements for control systems increased in scope. Modeling systems by using linear, time-invariant differential equations and subsequent transfer functions became inadequate. The ***state-space*** approach (also referred to as the ***modern***, or ***time-domain***, approach) is a unified method for modeling, analyzing, and designing a wide range of systems. For example, the state-space approach can be used to represent nonlinear systems that have backlash, saturation, and dead zone. Also, it can handle, conveniently, systems with nonzero initial conditions.

1. Transfer Function

For a linear time-invariant system with zero initial conditions, the transfer function is the ratio of the Laplace transform of the output to the Laplace transform of the input:

$$G(s) = Y(s)/U(s)$$

A general nth-order transfer function can be written as:

$$G(s) = (b_0 s^m + b_1 s^{m-1} + \cdots + b_m)/(s^n + a_1 s^{n-1} + a_2 s^{n-2} + \cdots + a_n)$$

The highest power of s in the denominator determines the system order after pole-zero cancellations. An nth-order minimal system normally requires n independent state variables.

2. State-Space Representation

The standard continuous-time state-space model is:

$$\dot{\boldsymbol{x}}(\boldsymbol{t}) = \boldsymbol{A}\boldsymbol{x}(\boldsymbol{t}) + \boldsymbol{B}\boldsymbol{u}(\boldsymbol{t})$$

$$\boldsymbol{y}(\boldsymbol{t}) = \boldsymbol{C}\boldsymbol{x}(\boldsymbol{t}) + \boldsymbol{D}\boldsymbol{u}(\boldsymbol{t})$$

where:

x(t)	State vector containing the internal state variables
A	System/state matrix: describes interaction among states
B	Input matrix: describes how the input affects the states
C	Output matrix: describes how states form the output
D	Direct feedthrough matrix: direct input-to-output path

3. Relationship Between Transfer Function and State Space

For the state-space equations, taking the Laplace transform with zero initial conditions gives:

$$sX(s) = AX(s) + BU(s)$$

$$(sI - A)X(s) = BU(s)$$

$$X(s) = (sI - A)^{-1}BU(s)$$

Substituting into the output equation gives:

$$Y(s) = C(sI - A)^{-1}BU(s) + DU(s)$$

$$G(s) = C(sI - A)^{-1}B + D$$

This equation shows why many different state-coordinate choices can produce exactly the same transfer function.

4. MATLAB Commands

The principal MATLAB commands used in this laboratory are:

tf(num,den)	% Create a transfer-function model
tf2ss(num,den)	% Convert numerator/denominator arrays to state-space matrices
ss(A,B,C,D)	% Create a state-space model
ss2tf(A,B,C,D)	% Convert state-space matrices to transfer-function coefficients
step(sys)	% Plot the unit-step response
pole(sys)	% Compute system poles

5. Converting Transfer Function to State Space.

We use the following MATLAB statements to form an LTI state-space representation from the transfer function of any arbitrary signal.

```
num = [1,7,2];
den = [1,9,26,24];
[A,B,C,D] = tf2ss(num,den)
% rearrange ordering of matrix and vector elements to put x1 on top:
P = [0,0,1;0,1,0;1,0,0]
A = inv(P) * A * P
B = inv(P) * B
C = C * P
```

5.1. MATLAB Conversion Using tf2ss

```
num = [1,7,2];
den = [1,9,26,24];
[A,B,C,D] = tf2ss(num,den)
```

MATLAB returns the following valid realization:

$$A = [-9 \ -26 \ -24; \ 1 \ 0 \ 0; \ 0 \ 1 \ 0]$$

$$B = [1; \ 0; \ 0]$$

$$C = [1 \ 7 \ 2]$$

$$D = 0$$

Therefore the state equations are:

$$\dot{x}_1 = -9x_1 - 26x_2 - 24x_3 + u$$

$$\dot{x}_2 = x_1$$

$$\dot{x}_3 = x_2$$

$$y = x_1 + 7x_2 + 2x_3$$

5.2. Why Reorder the State Variables?

The tf2ss realization is correct, but its internal states are ordered in a direction opposite to the commonly used phase-variable chain. From $\dot{x}_2 = x_1$ and $\dot{x}_3 = x_2$, reversing the state order produces the convenient sequence $\dot{x}_1 = x_2$ and $\dot{x}_2 = x_3$.

Use the permutation matrix:

$$P = [0\ 0\ 1;\ 0\ 1\ 0;\ 1\ 0\ 0]$$

This matrix reverses the first and third coordinates. For the new coordinate vector:

$$z = P^{-1}x$$

For this particular permutation matrix:

$$P^{-1} = P$$

$$P^2 = I$$

5.3. Coordinate Transformation

Starting with x = Pz and differentiating gives ẋ = Pż. Substituting into ẋ = Ax + Bu:

$$P\dot{z} = APz + Bu$$

$$\dot{z} = P^{-1}APz + P^{-1}Bu$$

Thus the transformed matrices are:

$$A_new = P^{-1}AP$$

$$B_new = P^{-1}B$$

$$C_new = CP$$

$$D_new = D$$

5.4. MATLAB State Reordering

```
% rearrange ordering of matrix and vector elements to put x1 on top:
P = [0,0,1;0,1,0;1,0,0];
A = inv(P) * A * P;
```

```
B = inv(P) * B;
C = C * P;
```

The resulting matrices are:

$$A = [0\ 1\ 0;\ 0\ 0\ 1;\ -24\ -26\ -9]$$

$$B = [0;\ 0;\ 1]$$

$$C = [2\ 7\ 1]$$

$$D = 0$$

5.5. Final Phase-Variable State Equations

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = x_3$$

$$\dot{x}_3 = -24x_1 - 26x_2 - 9x_3 + u$$

$$y = 2x_1 + 7x_2 + x_3$$

The transformed realization has the familiar controllable canonical/phase-variable structure. The input-output behavior is unchanged; only the internal state coordinates have changed.

Exercise Problem 1:

Problem Statement

For your assigned coefficient row, construct and analyze:

$$G(s) = (b_0s^2 + b_1s + b_2)/(s^3 + a_1s^2 + a_2s + a_3)$$

Determine the transfer function, poles, state-space matrices A, B, C and D, scalar state equations, transformed state-space realization, and verify that both realizations have identical input-output behavior.

Coefficient Allocation : Students (REGD # 1 to 45) choose coefficient from the list

Students (REGD # 46 to 90) choose coefficient from the list (1- 45) 90 onward gagain choose from the list (1-90)

Control System- CP-402

Variant	a_1	a_2	a_3	b_0	b_1	b_2
1	8	20	16	1	5	2
2	7	18	12	2	3	4
3	9	24	18	1	6	3
4	10	21	14	2	5	1
5	6	15	10	1	4	2
6	11	28	20	3	7	2
7	8	27	18	2	4	5
8	9	19	14	1	5	4
9	7	22	15	3	2	6
10	10	30	24	2	6	3
11	12	35	30	1	7	4
12	8	16	15	3	5	2
13	9	27	20	2	8	1
14	7	20	12	1	3	5
15	10	26	15	2	7	4
16	11	33	24	3	5	6
17	6	18	8	2	6	1
18	8	25	16	1	4	7
19	9	21	12	3	6	2
20	10	28	18	2	5	7
21	12	32	24	1	8	3
22	7	17	10	3	4	2
23	9	29	18	2	7	5
24	11	30	20	1	6	4
25	8	23	12	3	5	6
26	10	24	16	2	8	3
27	7	19	9	1	6	4
28	12	36	30	3	7	1
29	9	25	14	2	4	6
30	11	27	16	1	5	3
31	8	26	15	3	6	5
32	10	32	21	2	7	1
33	7	21	12	1	8	4
34	9	23	15	3	5	2
35	12	34	24	2	6	7
36	6	20	9	1	4	3
37	11	29	18	3	8	2
38	8	22	14	2	5	6
39	10	34	28	1	7	5
40	9	26	16	3	4	7
41	7	24	10	2	6	5
42	12	31	20	1	5	3
43	8	28	16	3	7	4

44	10	25	14	2	4	1
45	11	37	30	1	8	6

Required Tasks

1. Create num and den from your assigned row and display G(s) using tf.

2. Determine the system order and calculate the poles using pole(G).

3. Use [A,B,C,D] = tf2ss(num,den). Record the matrices.

4. Write the three scalar state equations and the output equation.

5. Explain the meaning of A, B, C and D.

6. Use the required permutation matrix P and calculate the transformed A, B and C matrices.

7. Write the transformed scalar state equations.

8. Construct both state-space models using ss.

9. Use ss2tf to recover the transfer function from the transformed model.

10. Plot both step responses on the same axes and verify equivalence.

11. Change a_3 by +1, rerun the model, and explain which canonical-form coefficient changes.

Exercise Problem 2: State Space to Transfer Function

Consider:

$$\dot{x}(t) = Ax(t) + Bu(t)$$

$$y(t) = Cx(t) + Du(t)$$

where

$$A = [0\ 1\ 0;\ 0\ 0\ 1;\ -24\ -26\ -9]$$

$$B = [0;\ 0;\ 1]$$

$$C = [2\ 7\ 1]$$

$$D = [0]$$

- **Task 1— Write the Scalar State Equations**

From A and B, write:

$$\dot{x}_1 = _____$$

$$\dot{x}_2 = _____$$

$$\dot{x}_3 = _____$$

Output equation:

$$y = _____$$

- **Task 2 — Manual Derivation of the Transfer Function**

Using zero initial conditions, derive:

$$G(s) = Y(s)/U(s)$$

Use:

$$G(s) = C(sI - A)^{-1}B + D$$

Calculate sI − A.
Calculate $(sI - A)^{-1}$, or solve for X(s) without explicitly forming the full inverse.
Calculate $(sI - A)^{-1}B$.
Premultiply by C.
Add D and simplify G(s).

- **Task 3 — MATLAB Verification**

```
clc;
clear;
close all;

A = ?
B = ?

C = ?

D = ?

% State-space to transfer function
[num,den] = ss2tf(A,B,C,D);

G = tf(num,den)
```

```
% Poles
p = pole(G)
```

Numerator: _____

Denominator: _____

- **Task 4 — Compare Manual and MATLAB Results**

$$G(s) = _____$$

Are the manual and MATLAB results identical? Explain.

_____

_____

- **Task 5 — Pole Analysis**

Determine the poles manually from the denominator and verify them using MATLAB.

Manual poles: _____

MATLAB poles: _____

- **Task 6 — Challenge Question**

Examine the last row of A:

$$A_3 = [-24 \quad -26 \quad -9]$$

Without performing the complete transfer-function calculation, explain how these coefficients are related to the characteristic polynomial and the denominator of the transfer function.

Answer:

- **Task 7 — Step-Response Verification**

```
sys_ss = ss(A,B,C,D);
sys_tf = tf(num,den);

figure;
step(sys_ss,sys_tf);
grid on;
legend('State-Space','Transfer Function');
title('Step Response Comparison');
```

Explain why the two responses should be identical.

_____

_____